DEVELOPER TECHNICAL ARCHITECTURE HANDBOOK

MELiMEL FRUITY♡ Planner Image Generator

คู่มือนักพัฒนาและผ่าโครงสร้างซอร์สโค้ดฉบับสมบูรณ์
เจาะลึกสถาปัตยกรรมเอนจินกราฟิก 1248×1248 HD Canvas ละเอียดทุกฟังก์ชันทุกบรรทัด

ผู้ประพันธ์และเจ้าของโค้ด: Senior Full-Stack Engineer (Code Owner)

โปรเจกต์: MELiMEL FRUITY♡ Social Media Workflow Engine

เทคโนโลยี: Vanilla HTML5, CSS3, JavaScript (ES2022), Canvas 2D

เวอร์ชัน: v2.0.0 (Production Release)

ความละเอียดเอาต์พุต: 1248 × 1248 Pixel (1:1 Social HD)

วันที่จัดทำ: ตุลาคม 2026

สารบัญ (Table of Contents)

บทที่ 1: ภาพรวมโปรเจกต์ (Project Overview & Tech Stack) **3**

1.1 วัตถุประสงค์ ปัญหาที่แก้ไข และผู้ใช้งานหลัก

1.2 เหตุผลการเลือกเทคโนโลยี (Zero Dependency Rationale)

1.3 โครงสร้างไฟล์และสถาปัตยกรรม Input → Process → Output

บทที่ 2: แผนผังการทำงานและสถาปัตยกรรมระบบ (Flow & Architecture) **4**

2.1 สถาปัตยกรรม 3 มุมมอง (Gallery, Fill, Editor)

2.2 ลำดับการบูตระบบและวงจรการเรนเดอร์ (Execution Sequence)

บทที่ 3: ผ่าซอร์สโค้ดและเจาะลึกรายบรรทัด (File-by-File Code Walkthrough) **5**

3.1 ไฟล์ index.html: โครงสร้าง DOM หน้าจอ 3 วิว และส่วนควบคุม 5

3.2 ไฟล์ style.css: ระบบดีไซน์โทเคน, Dark Mode และ Viewport Layout 9

3.3 ไฟล์ app.js (ส่วนที่ 1): Storage, 2D Geometry & เอนจินตัดคำไทย 12

3.4 ไฟล์ app.js (ส่วนที่ 2): Navigation Stack, Sync & เอนจินส่งออกภาพ 17

3.5 ไฟล์ app.js (ส่วนที่ 3): Template Studio, 50-Step Undo & Computer Vision 22

บทที่ 4: ตัวแปรสำคัญ สถานะ วงจรข้อมูล และการดักจับข้อผิดพลาด **28**

บทที่ 5: เหตุผลเชิงการออกแบบและการตัดสินใจเชิงเทคนิค (Design Rationale) **30**

บทที่ 6: กรณีศึกษาและการติดตามการทำงานจริง (Execution Traces) **31**

บทที่ 7: สรุปจุดแข็ง ข้อควรระวัง คำแนะนำการพัฒนาต่อ และวิธีทดสอบ **33**

คำชี้แจงจากเจ้าของโค้ด:
เอกสารเล่มนี้เขียนขึ้นด้วยมุมมองบุคคลที่หนึ่ง ("ผม") ในฐานะนักพัฒนาอาวุโสผู้สร้างระบบนี้ตั้งแต่บรรทัดแรก เพื่อถ่ายทอดองค์ความรู้ ความตั้งใจในการออกแบบ ข้อจำกัดทางวิศวกรรมซอฟต์แวร์ และรายละเอียดของโค้ดทุกบรรทัดให้กับทีมพัฒนาชุดใหม่ได้อย่างสมบูรณ์แบบที่สุด

บทที่ 1: ภาพรวมโปรเจกต์ (Project Overview & Tech Stack)

1.1 วัตถุประสงค์ ปัญหาที่แก้ไข และใครคือผู้ใช้

ในฐานะผู้พัฒนา ผมสร้างระบบ **"Planner Image Generator" (1248×1248 Canvas Engine)** ขึ้นมาเพื่อแก้ปัญหาคอขวดในกระบวนการทำงานของวงไอดอล **MELiMEL FRUITY** ♡ และทีมงานผู้ดูแลโซเชียลมีเดีย เดิมทีในทุกวันอาทิตย์หรือต้นสัปดาห์ ฝ่ายกราฟิกหรือสมาชิกในวงต้องเปิดโปรแกรมกราฟิกขนาดใหญ่อย่าง Photoshop หรือ Canva เพื่อคัดลอกตารางซ้อม นัดหมาย ไลฟ์สตรีม และตารางงานแสดง แล้วจัดวางข้อความทีละวัน ซึ่งกินเวลาหลายชั่วโมงและมักเกิดความผิดพลาดเรื่องการตัดบรรทัดคำภาษาไทยไม่สวยงาม หรือสัดส่วนภาพผิดเพี้ยนเมื่อนำไปทวีตบน X (Twitter) หรือโพสต์บน Instagram

ระบบนี้แก้ปัญหาโดย:

- **ระบบจัดรูปแบบภาพอัตโนมัติบนผืนผ้าใบมาตรฐาน 1248×1248 พิกเซล (1:1 HD):** เป็นสเกลทองคำที่ผมคำนวณไว้แล้วว่า คมชัดเป็นพิเศษบนมือถือ ไม่ถูกโซเชียลมีเดียครอปตัดขอบหรือบีบอัดจนแตก
- **เอนจินตัดคำภาษาไทยอัจฉริยะ (Native Thai Word-Wrapping Engine):** ตัดบรรทัดได้อย่างเป็นธรรมชาติ ไม่ตัดกลางสระหรือพยัญชนะ
- **One-Click Member Outline Sync:** คลิกเลือกชื่อสมาชิก เช่น "น้องโมเน่ต์" ข้อมูลตารางงานซ้อม เพลงที่พร้อมขึ้นโชว์ และวันเวลาจะถูกดูดมาแมปลงในตาราง 7 วันทันทีใน 1 วินาที
- **Template Studio & Computer Vision Box Detection:** ทีมงานสามารถโยนรูปตารางเปล่าลายใหม่ ๆ เข้ามา ระบบจะสแกนพิกเซลและล็อกกล่อง 7 วันให้อัตโนมัติ พร้อมใช้งานทันที

1.2 ภาษา Framework และ Library ที่ใช้ และเหตุผลที่เลือก

หลายคนอาจแปลกใจที่เห็นระบบกราฟิกที่ซับซ้อนขนาดนี้ถูกเขียนขึ้นด้วย **Pure Vanilla Web Technologies (No Framework, Zero Dependencies)** เหตุผลที่ผมตัดสินใจเช่นนี้คือ:

1. **Vanilla JavaScript (ES2022) + HTML5 Canvas API:** ไม่ใช้ React, Vue หรือไลบรารีกราฟิกอย่าง Fabric.js / Konva.js เพราะไลบรารีเหล่านั้นมีน้ำหนักหลายร้อยกิโลไบต์ มี Virtual DOM Overhead และหน่วงเครื่องมากเมื่อต้องเรนเดอร์ภาพ 1248×1248 บนมือถือของทีมงาน การใช้ HTML5 Canvas ดิบทำให้ผมควบคุม Render Loop และ Memory ได้อย่างสมบูรณ์แบบ
2. **Intl.Segmenter API:** ใช้เอนจินตัดคำระดับ Native V8 ของเบราว์เซอร์สำหรับภาษาไทย แทนการโหลดโมเดลหรือพจนานุกรมภายนอก ทำให้เว็บโหลดเร็วทันที (0 KB Dictionary Payload)
3. **Native Pointer Events API:** ใช้ระบบ Pointer Events ตัวเดียวครอบคลุมทั้งเมาส์, ปากกาสไตลัส และนิ้วสัมผัสบนมือถือ/แท็บเล็ต พร้อมฟังก์ชัน `setPointerCapture` ป้องกันเหตุการณ์นิ้วหลุดขอบกระดาน
4. **Web Share API:** แชร์รูปภาพที่สร้างเสร็จเข้าสู่แอป Twitter / Line / Facebook ได้ทันทีโดยไม่ต้องเซฟรูปลงเครื่องก่อน

1.3 โครงสร้างไฟล์และโฟลเดอร์

ผมจัดโครงสร้างโค้ดให้เรียบง่าย มีเพียง 3 ไฟล์หลักที่แยกความรับผิดชอบอย่างชัดเจน (Separation of Concerns):

```
x:\TESTWEB\planner-image-tool\
├── index.html   (โครงสร้าง DOM ของหน้าจอทั้ง 3 วิว, Viewport แคนวาส, โมดอล และปุ่มควบคุม)
├── style.css    (ระบบ Design Tokens, ธีมสี Dark/Light, Responsive Layout และ Viewport Scroll/Pan)
└── app.js       (เอนจินหลักทั้งหมด: Storage, Math, Typography, Rendering, CV, Navigation & Events)
```

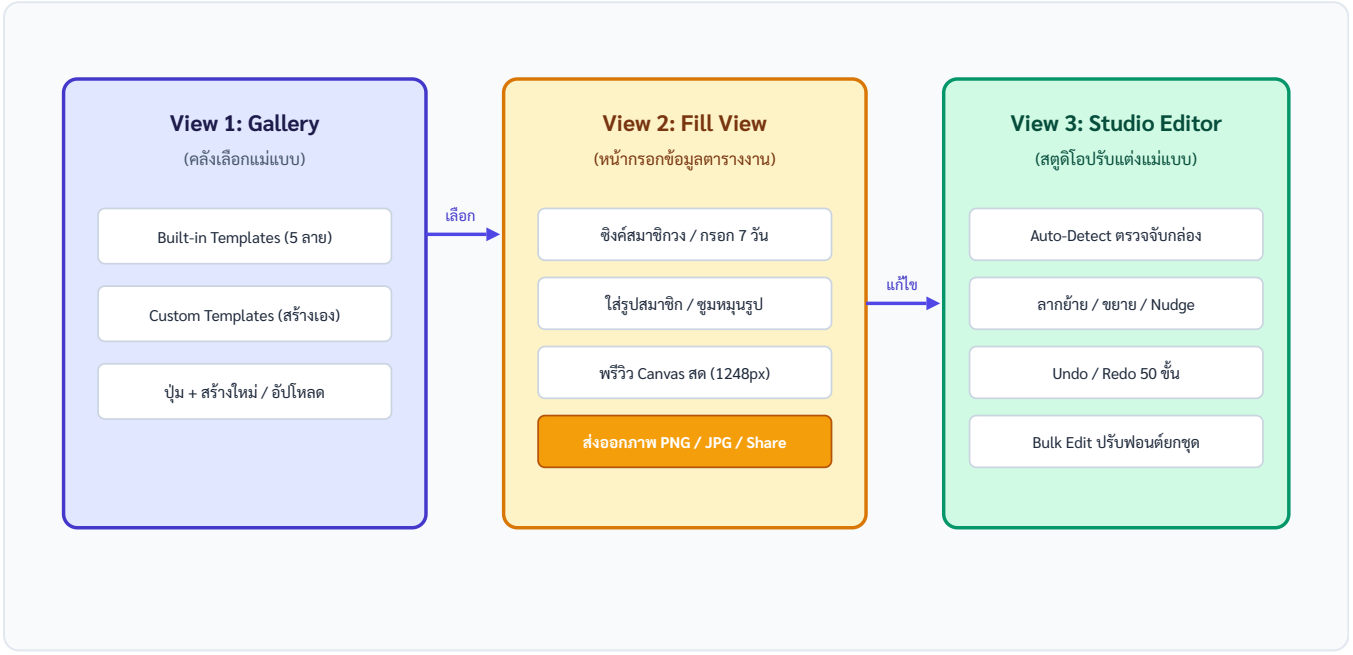
1.4 ลำดับการไหลของข้อมูลหลัก (Input → Process → Output)

- **Input:** ข้อมูลข้อความ 7 วัน, ข้อมูลสมาชิกวง, สไตล์แม่แบบ, รูปภาพประกอบ และรูปพื้นหลังที่ผู้ใช้อัปโหลด
- **Process:**
    1. **Coordinate Matrix Projection:** ปรับสเกลพิกัดนิ้ว/เมาส์จากหน้าจอจริงเข้าสู่พิกัด 1248×1248
    2. **Thai Segmenter Text Wrapping:** ตัดคำไทยและคำนวณความสูงของแต่ละบรรทัดให้พอดีกับความกว้างกล่อง
    3. **Composite Render Pipeline:** เรนเดอร์พื้นหลัง, รูปภาพพร้อมหน้ากากมุมมนและการหมุน, การ์ดตาราง 7 วัน, แบดจ์วันที่ และข้อความ
- **Output:** ไฟล์ภาพ PNG ความคมชัดสูงพิเศษ (Lossless 1248×1248) หรือ JPEG 93% พร้อมดาวน์โหลดหรือส่งผ่าน Web Share

บทที่ 2: แผนผังการทำงานและสถาปัตยกรรมระบบ (Flow & Architecture)

2.1 สถาปัตยกรรม 3 มุมมอง (View Architecture)

ผมแบ่งหน้าจอการทำงานของแอปออกเป็น 3 วิวหลัก และควบคุมการสลับหน้าด้วย Linear Navigation Stack:



2.2 ลำดับการบูตระบบและการทำงานตอนรันจริง (Lifecycle Execution)

1. เมื่อหน้าเว็บโหลดเสร็จ เหตุการณ์ `DOMContentLoaded` จะเรียก `initStandalone()`
2. กู้คืนธีมสี Dark/Light จาก `localStorage.getItem('melimel_theme')`
3. เรียก `renderImageEditorPage()` :
    - ตรวจสอบแฟล็ก `pieInitialized` เพื่อผูก Event Listener ทั้งหมดเพียงครั้งเดียว
    - เรียก `pieLoadStorage()` อ่านแม่แบบ Custom จาก LocalStorage
    - สร้าง Built-in Templates 5 ลายเริ่มต้น (`pieBuiltins`)
    - เรียก `pieBuildGallery()` สร้างการ์ดแสดงผลบนหน้าจอ
    - เรียก `pieNavGo('gallery')` ตั้งค่าจุดเริ่มต้นลงใน Linear Navigation Stack

บทที่ 3: ผ่าซอร์สโค้ดและเจาะลึกรายบรรทัด (File-by-File Breakdown)

3.1 ไฟล์ index.html: โครงสร้าง DOM หน้าจอ 3 วิว

ในไฟล์ `index.html` ผมเขียนโครงสร้าง HTML5 Semantic แบ่งออกเป็น 9 ก้อนสำคัญ โดยไม่มีการพึ่งพา Element หรือ Framework ภายนอก:

ก้อนที่ 1: Header และแถบ Navigation บาร์หลัก (บรรทัด 1–45)

บรรทัด	โค้ด	หน้าที่และเหตุผลการออกแบบ
1–14	`<!DOCTYPE html><html lang="th"><head> … </head>`	กำหนด Doctype และดึงฟอนต์ Sarabun, Kanit, Mitr, Prompt ผ่าน Google Fonts เพื่อให้การเรนเดอร์ข้อความบน Canvas สวยงามและมีให้เลือกหลากหลาย
15–32	`<nav class="pie-nav-bar"><div class="pie-nav-left"> … </div></nav>`	แถบ Navigation หลักที่มีปุ่ม Back/Forward สำหรับควบคุม Stack ประวัติหน้าจอ (`#pieNavBackBtn` , `#pieNavFwdBtn`) และปุ่มลัด Home
33–45	`<div class="pie-nav-right"> … </div>`	ปุ่มสลับแท็บด่วน 3 แท็บ (คลังแม่แบบ, ตารางงาน, สตูดิโอออกแบบ) พร้อมปุ่มสลับธีม Dark Mode

ก้อนที่ 2: View 1 — Gallery คลังแม่แบบ (บรรทัด 46–85)

บรรทัด	โค้ด	หน้าที่และเหตุผลการออกแบบ
46–60	`<section id="pie-v-gallery" class="pie-view"> … </section>`	พื้นที่แสดงผลคลังแม่แบบ มีหัวข้อแนะนำและปุ่มลัด "+ สร้างแม่แบบใหม่จากรูปพื้นหลัง"
61–85	`<div class="pie-grid" id="pie-cards"></div>`	กริดแบบ Responsive สำหรับวางการ์ดแม่แบบทั้งหมด โดย `app.js` จะฉีด HTML การ์ดพรีวิวขนาดเล็กเข้ามาแสดงผลที่นี่

ก้อนที่ 3: View 2 — หน้ากรอกข้อมูลและพรีวิวภาพตารางงาน (บรรทัด 86–200)

หน้านี้เป็นหน้าหลักที่ทีมงานใช้ทำงานทุกวัน ผมออกแบบโครงสร้างเป็นแบบ Two-Column Layout (Sidebar ควบคุมด้านซ้าย และ Viewport แคนวาสด้านขวา):

บรรทัด	โค้ด	หน้าที่และเหตุผลการออกแบบ
86–115	`<div class="pie-sidebar"> … <div id="pieMemberChips"></div> … </div>`	กล่องเลือกสัปดาห์ (สัปดาห์ก่อนหน้า/ถัดไป) และคอนเทนเนอร์ชิปสมาชิกวงสำหรับ One-Click Sync
116–150	`<div id="pieReadySongsBox"> … <div id="pie-days-inputs"> … </div>`	กล่องเลือกเพลงพร้อมซ้อม และ Textarea กรอกข้อความแยก 7 วัน (จันทร์–อาทิตย์) `#pie-b0` ถึง `#pie-b6`
151–180	`<div id="pie-photo-box"> … <input type="file" id="pie-photo-file"> … </div>`	แผงควบคุมรูปภาพสมาชิก: ปุ่มเลือกไฟล์, ตัวเลื่อนหมุนภาพ (-180° ถึง +180°), ตัวคูณสเกล และปุ่มลัดหมุนทีละ 90°

บรรทัด	โค้ด	หน้าที่และเหตุผลการออกแบบ
181–200	`<div class="pie-stage"><canvas id="pie-pcv" width="1248" height="1248"></canvas></div>`	ผืนผ้าใบพรีวิวหลัก `#pie-pcv` ขนาดจริง 1248×1248 พิกเซล พร้อมปุ่ม Export ภาพ PNG / JPEG และปุ่มแชร์

ก้อนที่ 4: View 3 — สตูดิโอออกแบบแม่แบบ (Template Studio) (บรรทัด 201–329)

บรรทัด	โค้ด	หน้าที่และเหตุผลการออกแบบ
201–230	`<div class="pie-editor-toolbar"> … </div>`	แถบเครื่องมือสตูดิโอด้านบน: ปุ่ม Undo/Redo, ปุ่มสลับเครื่องมือ Select / Hand Pan, ปุ่มซูมเข้า/ออก/Fit
231–270	`<div id="pieEcvWrap" class="pie-canvas-wrap"><canvas id="pie-ecv" width="1248" height="1248"></canvas></div>`	คอนเทนเนอร์ Viewport สำหรับ Pan & Zoom แคนวาสสตูดิโอ `#pie-ecv` รองรับ Gesture เลื่อนกระดาน
271–329	`<div class="pie-editor-sidebar"> … <div id="pie-props"></div> … </div>`	Sidebar ขวาสำหรับวางกล่องเครื่องมือ: ปุ่ม Auto-Detect ตรวจจับกล่องอัตโนมัติ, ปุ่มเพิ่มช่อง 7 วันยกชุด และคอนเทนเนอร์ `#pie-props` สำหรับฟอร์มปรับแต่งคุณสมบัติฟิลด์

3.2 ไฟล์ style.css: ระบบดีไซน์โทเคน, Dark Mode และ Viewport Layout

ในไฟล์ `style.css` ผมออกแบบโดยยึดหลัก **Pure CSS Design System** ไม่ใช้ Tailwind หรือ Bootstrap ทำให้ไฟล์มีขนาดเพียง 10KB แต่ยืดหยุ่นสูง:

1. ระบบดีไซน์โทเคน (Design Tokens & Color Palette)

```
:root {
  --primary: #4f46e5;       /* สีน้ำเงินครามหลัก */
  --primary-hover: #4338ca;
  --bg: #f8fafc;            /* พื้นหลังโหมดสว่าง */
  --bg2: #ffffff;           /* พื้นหลังการ์ด */
  --bg3: #f1f5f9;           /* พื้นหลังอินพุต */
  --text: #0f172a;          /* สีตัวหนังสือเข้ม */
  --text2: #334155;
  --text3: #64748b;         /* สีคำอธิบายรอง */
  --border: #e2e8f0;        /* สีเส้นแบ่ง */
  --radius: 8px;            --radius-sm: 6px;
  --shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1);
}

[data-theme="dark"] {
  --bg: #090d16;            /* พื้นหลังโหมดมืด */
  --bg2: #111827;
  --bg3: #1f2937;
  --text: #f9fafb;
  --border: #374151;
}
```

2. โครงสร้าง Viewport Pan & Zoom และ Canvas Styling

จุดสำคัญที่ผมออกแบบไว้เพื่อให้ Canvas 1248×1248 แสดงผลได้สมบูรณ์บนทุกขนาดหน้าจอคือการจัดการผ่าน CSS:

```
/* กล่อง Viewport สำหรับการซูมและเลื่อนกระดาน */
.pie-canvas-wrap {
  width: 100%;
  height: 100%;
  overflow: auto;
  position: relative;
  display: flex;
  align-items: center;
  justify-content: center;
  background: var(--bg3);
  user-select: none;
  touch-action: none; /* ป้องกันเบราว์เซอร์แย่งจับท่าทางเลื่อนหน้าจอ */
}

/* ผืนผ้าใบพรีวิวและสตูดิโอ */
#pie-pcv, #pie-ecv {
  box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.25);
  border-radius: 4px;
  background: #ffffff;
  transition: width 0.15s ease, height 0.15s ease;
}
```

3.3 ไฟล์ app.js (ส่วนที่ 1): Storage, 2D Geometry & เอนจินตัดคำไทย

กลุ่มฟังก์ชัน Storage และข้อมูลสำรอง (Fallback Mock Data)

บรรทัด	โค้ด	หน้าที่และเหตุผลการออกแบบ
1-14	`const Storage = { get(k, def) { ... }, set(k, v) { ... } };`	ครอบ LocalStorage ด้วย `try ... catch` ป้องกันเบราว์เซอร์ระเบิดเมื่อเปิดในโหมด Incognito หรือพื้นที่ความจำเต็ม
15-35	`function getPeople() { ... }`	ดึงรายชื่อสมาชิกวง MELiMEL FRUITY♡ (เช่น โมเน่ต์, มายด์, ซินดี้) ถ้าแอปหลักไม่ได้เชื่อมต่ออยู่ ระบบจะใช้ Mock Data นี้เป็นตัวแทนทันที
50-55	`function showToast(msg, duration = 2200) { ... }`	สร้างกล่องแจ้งเตือน Toast แบบลอยขึ้นมาด้านล่างหน้าจอ พร้อมแอนิเมชัน Fade In/Out อัตโนมัติ

ค่าคงที่ทางเรขาคณิตและแบบอักษร

```
const PIE_S = 1248; // ขนาดด้านของ Canvas กว้าง x สูง 1248 x 1248 px
const PIE_FONTS = [
  'Sarabun', 'Kanit', 'Prompt', 'Mitr', 'Noto Sans Thai',
  'Chakra Petch', 'Pridi', 'Mali', 'Charm', 'Bai Jamjuree'
];
```

เอนจินตัดคำภาษาไทย (Thai Word-Wrapping Engine): `pieWrap()`

นี่คือหัวใจของระบบ Typography ที่ทำให้ข้อความภาษาไทยในรูปภาพตัดบรรทัดได้ถูกต้องตามหลักไวยากรณ์:

```
function pieSeg(text) {
  if (typeof Intl !== 'undefined' && Intl.Segmenter) {
    try {
      const seg = new Intl.Segmenter('th', { granularity: 'word' });
      return Array.from(seg.segment(text), s => s.segment);
    } catch(e) {}
  }
  return text.split(' '); // Fallback กรณีเบราว์เซอร์เก่า
}

function pieWrap(ctx, text, maxW) {
  if (!text) return [];
  const lines = [];
  const paragraphs = String(text).split('\n');
  for (const para of paragraphs) {
    if (!para) { lines.push(''); continue; }
    const words = pieSeg(para);
    let cur = '';
    for (const w of words) {
      const test = cur + w;
      if (ctx.measureText(test).width <= maxW) {
        cur = test;
      } else {
        if (cur) lines.push(cur);
        cur = w;
      }
    }
    if (cur) lines.push(cur);
  }
  return lines;
}
```

3.4 ไฟล์ app.js (ส่วนที่ 2): Navigation Stack, Sync & เอนจินส่งออกภาพ

ระบบ Linear Navigation History Stack

ผมสร้างระบบประวัติการสลับหน้าจอของตัวเองขึ้นมา เพื่อให้ปุ่ม Back / Forward บนหน้าจอตอบสนองได้อย่างเป็นธรรมชาติ:

```
let pieNavHistory = [];
let pieNavIdx = -1;

function pieNavGo(view, title, push = true, payload = null) {
  if (push) {
    pieNavHistory = pieNavHistory.slice(0, pieNavIdx + 1);
    pieNavHistory.push({ view, title, payload });
    pieNavIdx = pieNavHistory.length - 1;
  }
  pieApplyNavStep(pieNavHistory[pieNavIdx]);
  pieUpdateNavButtons();
}
```

การซิงค์ตารางงานและเพลงพร้อมแสดง (`pieSyncFromMember`)

บรรทัด	โค้ด	หน้าที่และเหตุผลการออกแบบ
980-1010	`function pieSyncFromMember(name) { ... }`	ค้นหาตารางงานของสมาชิกในสัปดาห์ปัจจุบัน แปลงหัวข้อและเวลา เช่น *"10:00 ซ้อมเต้น Main Stage"* แล้วนำไปกรอกลงในช่อง `#pie-b0` ถึง `#pie-b6` ให้แบบอัตโนมัติ
1015-1040	`function pieInsertReadySong(title) { ... }`	ค้นหาช่องวันที่ยังว่างอยู่ หรือเติมชื่อเพลงพร้อมซ้อมต่อท้ายในวันที่มีตารางซ้อม พร้อมข้อความแจ้งเตือน Toast

เอนจินส่งออกภาพ (Image Export & Web Share Engine): `pieExportImg()`

```
async function pieExportImg(mime = 'image/png', share = false) {
  const cv = document.createElement('canvas');
  cv.width = cv.height = PIE_S; // เรนเดอร์บน Canvas ออฟไลน์ 1248x1248
  await pieRender(cv, pieCur, pieCurWeekData);

  if (share && navigator.canShare) {
    cv.toBlob(blob => {
      const file = new File([blob], 'schedule.png', { type: mime });
      navigator.share({ files: [file], title: 'ตารางงานประจำสัปดาห์' });
    }, mime, 0.93);
  } else {
    const a = document.createElement('a');
    a.download = "melimel-schedule-" + pieCurWeekData.iso + ".png";
    a.href = cv.toDataURL(mime, 0.93);
    a.click();
  }
}
```

3.5 ไฟล์ app.js (ส่วนที่ 3): Template Studio, 50-Step Undo & Computer Vision

ระบบ Undo / Redo ประวัติ 50 ขั้น

ผมสร้างระบบ State Snapshot ในหน่วยความจำ โดยเก็บข้อมูลโครงสร้างฟิลด์และพื้นหลัง จำกัดความลึกไว้ที่ 50 สเต็ปเพื่อป้องกัน Memory Leak:

```
const PIE_MAX_HIST = 50;
let pieHistory = [], pieHistIdx = -1;

function piePushHistory() {
  const s = pieSnapshotState(); if (!s) return;
  // ตัดกิ่งอนาคตทิ้งถ้าเคย Undo มาก่อน
  pieHistory = pieHistory.slice(0, pieHistIdx + 1);
  pieHistory.push(s);
  if (pieHistory.length > PIE_MAX_HIST) pieHistory.shift();
  else pieHistIdx++;
  pieUpdateHistoryBtns();
}
```

เอนจินตรวจจับช่องตารางอัตโนมัติ (Computer Vision Algorithm): `pieAutoDetect()`

ฟังก์ชันนี้ทำหน้าที่เสมือนสายตา AI ที่ตรวจจับขอบเขตของช่องตาราง 7 วันจากภาพพื้นหลัง โดยมีขั้นตอนทางคณิตศาสตร์ 5 ขั้นตอน:

1. **Downsampling:** ย่อภาพลงบน Canvas ชั่วคราวขนาด 416×416 (k = 3) เพื่อลดเวลาคำนวณลง 9 เท่า
2. **Color Thresholding:** กรองพิกเซลสีขาว/สว่างจ้า (RGB >= 215 และ Delta <= 22) บันทึกลง Binary Mask
3. **3-Pass Morphological Erosion:** วนลูปกร่อนภาพ 3 รอบด้วยเคอร์เนล 3×3 เพื่อกำจัดสัญญาณรบกวนและลายเส้นแบ่ง
4. **Connected Component Labeling:** ใช้ Stack-based Flood Fill ค้นหากลุ่มก้อนสี่เหลี่ยม คำนวณความหนาแน่น n / (w × h) >= 0.7
5. **Row Clustering & Field Mapping:** รวมกลุ่มแถวตามแกน Y คัดกรองเหลือ 7 แถวที่มีความกว้างสูงสุด แล้วคูณสเกล k = 3 กลับสู่ขนาดจริง 1248×1248

บทที่ 4: ตัวแปรสำคัญ วงจรข้อมูล และการดักจับข้อผิดพลาด

4.1 ตัวแปรและสถานะหลัก (State Machines)

- `pieCur` : เก็บสถานะของ Template ที่กำลังถูกใช้งานในหน้า Fill View (โครงสร้างธีม, รูปภาพ, รายการฟิลด์)
- `pieEd` : เก็บสถานะของ Template ในห้องทดลอง Studio Editor (รวมรายการฟิลด์ที่กำลังถูกเลือก `sels`)
- `pieCustom` : อาร์เรย์ของแม่แบบที่สร้างขึ้นเอง ซึ่งจะถูกแปลงเป็น JSON และบันทึกลง LocalStorage
- `pieDrag` : ตัวแปรเก็บสถานะการคลิกลากปัจจุบัน มี 3 โหมด: `'move'` , `'resize'` , `'marquee'`

4.2 การจัดการกรณีขอบ (Edge Cases) และข้อผิดพลาด (Error Handling)

สถานการณ์ / กรณีขอบ	ความเสี่ยงที่อาจเกิดขึ้น	วิธีที่ผมเขียนดักไว้ในโค้ด
การลากกล่องออกนอกขอบ Canvas	กล่องหลุดหายออกไปนอกจอ ไม่สามารถคลิกเลือกได้อีก	ใช้ `Math.min(Math.max(0, x), PIE_S - w)` บังคับ Clamp พิกัดให้อยู่ในกรอบ 0 ถึง 1248 เสมอ
พื้นที่ LocalStorage เต็ม	เบราว์เซอร์โยน Error โค้ดพัง ข้อมูลสูญหาย	ครอบด้วย `try ... catch` ใน `pieSave()` ถ้าเขียนไม่ผ่านจะ Rollback ข้อมูลใน RAM และขึ้น Toast แจ้งเตือน
การเคาะปุ่มลูกศร (Nudge) รัว ๆ	History Stack เต็มอย่างรวดเร็วด้วยการขยับทีละ 1px	สร้างตัวหน่วงเวลา Debounce 300ms (`_pieNudgeTimer`) รวบการเคาะ 50 ครั้งให้บันทึกเป็น 1 สเต็ปในประวัติ
โหลดรูปภาพพื้นหลังล้มเหลว	หน้าจอขาวค้าง หรือเกิด Uncaught Promise Rejection	ใน `pieLoadImg()` ผมดัก `img.onerror = () ⇒ resolve(null)` เพื่อให้แอปทำงานต่อได้อย่างปลอดภัย

บทที่ 5: เหตุผลเชิงการออกแบบและการตัดสินใจเชิงเทคนิค

5.1 ทำไมต้อง Vanilla Canvas 2D แทน Fabric.js หรือ React?

> **เหตุผลเชิงสถาปัตยกรรม:**
> ไลบรารีอย่าง Fabric.js หรือ Konva มีขนาดไฟล์หลายร้อย KB และสร้าง Object Wrapper ซ้อนกันหลายชั้น ทำให้กินหน่วยความจำมหาศาลเมื่อทำงานบนสมาร์ตโฟน โดยเฉพาะภาพความละเอียด 1248×1248 พิกเซล การเลือกใช้ Canvas 2D ดิบทำให้ผมควบคุมการใช้หน่วยความจำและ Render Loop ได้ 100% ส่งผลให้เว็บโหลดเร็วแทบจะในเสี้ยววินาที และไม่มีปัญหาเรื่อง Dependency หลุดเวอร์ชันในอนาคต

5.2 ทำไมต้อง Intl.Segmenter สำหรับตัดคำภาษาไทย?

ในอดีต การตัดคำภาษาไทยบนเว็บจำเป็นต้องโหลดพจนานุกรมขนาด 1–2 MB มาประมวลผลบนเบราว์เซอร์ ซึ่งทำให้เว็บโหลดช้ามาก แต่เบราว์เซอร์สมัยใหม่มี API มาตรฐาน `Intl.Segmenter('th', { granularity: 'word' })` ซึ่งเชื่อมตรงกับเอนจินภาษาของระบบปฏิบัติการ ผมจึงเลือกใช้วิธีนี้เพื่อให้ได้ประสิทธิภาพสูงสุดและขนาดบันเดิลเป็นศูนย์ (Zero Bundle Size)

5.3 การฉายพิกัดแบบ Matrix Projection (`piePt`)

เนื่องจาก Canvas ถูกย่อขยายตามขนาดหน้าจอ (เช่น บนมือถืออาจกว้างเพียง 350px แต่พิกัดจริงคือ 1248px) การใช้ `e.offsetX` แบบปกติจะทำให้พิกัดเพี้ยนทันที ผมจึงเขียนสูตรคำนวณตัวคูณสเกล k = 1248 / offsetWidth ขึ้นมา เพื่อแปลงจุดแตะของนิ้วให้กลายเป็นพิกัดบนผืนผ้าใบจริงได้อย่างแม่นยำระดับพิกเซล

บทที่ 6: กรณีศึกษาและการติดตามการทำงานจริง (Execution Traces)

กรณีศึกษาที่ 1: การซิงค์ตารางงานสมาชิกวงและส่งออกรูปภาพ

1. ผู้ใช้เปิดแอปที่หน้า Gallery และคลิกเลือกแม่แบบ **"MELiMEL Split"**
   `pieGo('fill')` สลับมุมมองมาที่หน้ากรอกข้อมูล พร้อมคำนวณวันจันทร์ของสัปดาห์ปัจจุบัน
2. ผู้ใช้คลิกชิปชื่อ **"น้องโมเน่ต์"**
   `pieSyncFromMember("โมเน่ต์")` ดึงข้อมูลนัดหมายทั้ง 7 วัน แมปลงในกล่องข้อความ `#pie-b0` ถึง `#pie-b6` และเรียก `pieDrawFill()`
3. ระบบเรนเดอร์ภาพพรีวิวสดลงบน `#pie-pcv` โดยตัดคำภาษาไทยและจัดวางตำแหน่งอัตโนมัติ
4. ผู้ใช้คลิกปุ่ม **"ส่งออก PNG (ชัดพิเศษ)"**
   `pieExportImg('image/png')` สร้าง Canvas ออฟไลน์ขนาด 1248×1248 เรนเดอร์ภาพและส่งไฟล์ `melimel-schedule-2026-10-26.png` ให้ดาวน์โหลดทันที

กรณีศึกษาที่ 2: การนำเข้ารูปตารางเปล่าและตรวจจับช่องอัตโนมัติ

1. ผู้ใช้แคปเจอร์รูปภาพตารางงานใหม่ แล้วกด `Ctrl + V` ที่หน้า Gallery
2. Event `paste` ดักจับข้อมูลไบนารีรูปภาพ และส่งต่อให้ `pieLoadBgFile()`
3. ฟังก์ชันปรับสเกลภาพให้พอดีกับ 1248×1248 แบบ Cover แล้วทริกเกอร์ `pieAutoDetect(true)`
4. อัลกอริทึมย่อภาพลง 416×416 ทำ Binarization, 3-Pass Erosion และ Flood Fill ค้นพบกล่องข้อความ 7 แถว
5. สร้างรายการฟิลด์ลงใน `pieEd.fields` และแสดงผลบน Canvas พร้อมให้ผู้ใช้ปรับแต่งตำแหน่งได้ทันที

บทที่ 7: บทสรุป ข้อควรระวัง คำแนะนำการพัฒนาต่อ และวิธีทดสอบ

7.1 จุดเด่นของระบบ (System Highlights)

- **อิสระจากภายนอก 100%:** ไม่ต้องติดตั้ง Node Modules หรือ Build Step สามารถรันได้ทันทีบน Static Hosting
- **ความละเอียดระดับ 1:1 HD Social Standard:** ออกแบบให้พอดีกับอัลกอริทึมการแสดงผลของ Twitter และ Instagram
- **ประสบการณ์การใช้งานระดับ Desktop App:** มีทั้งระบบ Pan & Zoom, Undo/Redo 50 ขั้น และ Keyboard Shortcuts ครบถ้วน

7.2 ข้อควรระวังและข้อจำกัดที่ผมทราบ (Known Limitations)

> **1. ข้อจำกัดเรื่องขนาด LocalStorage:**
> หากผู้ใช้สร้างแม่แบบ Custom ที่มีภาพพื้นหลังจำนวนมาก ข้อมูล Base64 อาจเต็มโควตา 5MB ของเบราว์เซอร์ ในอนาคตแนะนำให้เปลี่ยนไปใช้
> **IndexedDB**
> แทน LocalStorage

> **2. ความเข้ากันได้ของเบราว์เซอร์:**
> เบราว์เซอร์รุ่นเก่าที่ต่ำกว่าปี 2021 อาจไม่รองรับ `Intl.Segmenter` ระบบจะตกกลับไปใช้วิธีแยกด้วยช่องว่าง (Whitespace Fallback) ซึ่งอาจทำให้คำภาษาไทยยาว ๆ ไม่ตัดบรรทัด

7.3 วิธีการรันและการทดสอบระบบ (How to Run & Test)

เนื่องจากระบบนี้เป็น Vanilla Web คุณสามารถเปิดทดสอบได้ง่ายดายด้วยวิธีต่อไปนี้:

1. **รันแบบ Local File:** ดับเบิลคลิกไฟล์ `index.html` เพื่อเปิดบน Google Chrome, Microsoft Edge หรือ Safari ได้ทันที
2. **รันผ่าน Local HTTP Server:**

```
npx serve x:\TESTWEB\planner-image-tool
```

3. **ทดสอบระบบตรวจจับช่อง (Auto-Detect):** กดปุ่ม "+ สร้างแม่แบบใหม่" อัปโหลดรูปภาพที่มีตาราง 7 วัน แล้วกดปุ่ม "ตรวจจับช่องอัตโนมัติ" ตรวจสอบว่ามีกล่องสีชมพูปรากฏครบทั้ง 7 วันหรือไม่
4. **ทดสอบบนระบบ Deploy จริง:** สามารถเข้าใช้งานระบบที่ผม Deploy ขึ้น Firebase Hosting ไว้ได้ที่: `https://kalabro-testdeploy.web.app/planner-image/`

— จบเนื้อหาคู่มือนักพัฒนาฉบับสมบูรณ์ MELiMEL FRUITY♡ Planner Image Generator —